PROJECT REPORT

Script-Controlled ACL – Restrict Record Access Based on Field Value

Team ID: 6aba117e538da4003d63184c
Date: 03 October 2026

---

1. INTRODUCTION

1.1 Project Overview

The project **"Script-Controlled ACL – Restrict Record Access Based on Field Value"** focuses on implementing secure and flexible record-level access control using Access Control Lists (ACLs) in the ServiceNow platform.

Access Control Lists are used to control whether users are permitted to perform operations such as **Read, Create, Write, and Delete** on records. While role-based ACLs provide a basic level of authorization, some applications require access decisions based on the values stored within individual records.

This project addresses this requirement by combining **user roles, ACL conditions, and server-side JavaScript scripts**. The script validates the value of a specific field, such as the **Branch** field, before granting access to a record.

The project uses a custom table named **u_institution_details**, which contains institution-related records and a Branch field. Users with appropriate roles are permitted to perform operations according to the configured ACL rules, while users attempting to access restricted branch records are denied access.

The project demonstrates how ServiceNow ACLs can provide centralized, secure, and customizable authorization for enterprise applications.

1.2 Purpose

The primary purpose of this project is to demonstrate the implementation of **script-controlled, field-based access restrictions** using ServiceNow ACLs.

The project aims to:

- Implement READ, CREATE, WRITE, and DELETE access controls.
- Create and manage users and roles.
- Configure a custom u_institution_details table.
- Apply role-based authorization using ServiceNow ACLs.
- Implement server-side JavaScript for additional access validation.

- Restrict access according to the value of the Branch field.
- Prevent unauthorized users from viewing or modifying restricted records.
- Demonstrate secure record-level authorization within ServiceNow.

---

2. IDEATION PHASE

2.1 Problem Statement

Traditional role-based access control is sometimes insufficient when access to a record depends on the **data contained within that record**.

For example, a user may have permission to access institution records but should only be allowed to view or modify records belonging to a particular branch. Granting unrestricted table-level access could expose records belonging to other branches.

Therefore, the problem addressed by this project is:

"How can ServiceNow restrict users from accessing institution records based not only on their roles, but also on the value of a specific field such as Branch?"

The proposed project solves this problem by combining ServiceNow ACLs with server-side scripts that validate the Branch value before allowing the requested operation.

2.2 Empathy Map Canvas

User/Administrator Perspective

Category	Observation
Says	"Users should only access the records they are authorized to handle."
Thinks	"Role permissions alone may not be sufficient for branch-specific restrictions."
Does	Creates users, assigns roles, configures ACLs, creates records and tests permissions.
Feels	Concerned about unauthorized access and accidental modification of records.
Sees	Institution records, user roles, ACL configurations and access-denied messages.
Hears	Requirements from administrators regarding branch-specific access.

Pain Points	Unauthorized record access, excessive permissions and difficulty controlling record-level access.
Needs	Secure, centralized and configurable access control.

2.3 Brainstorming

During brainstorming, several possible approaches for controlling record access were considered.

Idea	Description
Role-Based Access	Use ServiceNow roles to control access to the table.
Table-Level ACL	Restrict access to the entire u_institution_details table.
Field-Based ACL	Control access according to specific field values.
Script-Controlled ACL	Use server-side JavaScript to dynamically evaluate access.
Branch-Based Restriction	Allow users to access records according to their authorized Branch.
Combined Approach	Combine roles, ACL conditions and scripts for stronger authorization.

The **Combined Approach** was selected because it provides both role-based and record-based security while remaining configurable within the ServiceNow platform.

---

3. REQUIREMENT ANALYSIS

3.1 Customer Journey Map

Stage	User Action	System Response	User Expectation
Login	User logs into ServiceNow	User identity is authenticated	Secure login
Role Verification	User accesses institution records	System identifies assigned roles	Correct permissions
Record Access	User requests a record	ACL engine evaluates READ permission	Authorized records displayed
Branch Validation	Script checks Branch value	Access is allowed or denied	Branch-specific restriction

Create	User attempts to create a record	CREATE ACL is evaluated	Creation allowed only for authorized users
Update	User modifies a record	WRITE ACL and script are evaluated	Unauthorized updates prevented
Delete	User deletes a record	DELETE ACL is evaluated	Unauthorized deletion prevented
Completion	User finishes operation	System records the permitted action	Secure operation

3.2 Solution Requirement

Functional Requirements

FR-1: User and Role Management

The system shall allow administrators to create users and assign appropriate ServiceNow roles.

FR-2: Table and Record Management

The system shall provide a custom u_institution_details table for storing institution records.

FR-3: Access Control Configuration

The system shall provide ACLs for READ, CREATE, WRITE and DELETE operations.

FR-4: Script-Based Access Restriction

The system shall execute server-side JavaScript during ACL evaluation to validate the Branch field and determine whether access should be granted.

Non-Functional Requirements

NFR-1: Security
The system must prevent unauthorized users from accessing restricted records.

NFR-2: Usability
ACL configuration and testing should be understandable to administrators.

NFR-3: Reliability
Access decisions should consistently follow the configured ACL rules.

NFR-4: Performance
ACL scripts should perform minimal processing to avoid unnecessary delays.

NFR-5: Availability
The access-control functionality should be available whenever the ServiceNow instance is available.

NFR-6: Scalability
The solution should support additional users, roles, branches and access rules without requiring a complete redesign.

3.3 Data Flow Diagram

Level 0 DFD

User → ServiceNow ACL System → u_institution_details

The ServiceNow ACL System performs:

1. User authentication
2. Role verification
3. ACL evaluation
4. Branch-based script validation
5. Access decision

The final decision is either:

- **Access Granted**, or
- **Access Denied**

Level 1 DFD

User

↓

1.0 User & Role Verification

↓

2.0 ACL Evaluation

↓

3.0 Script-Based Branch Validation

↓

4.0 Access Decision

↓

Allow → u_institution_details

Deny → Access Denied

3.4 Technology Stack

Component	Technology
Platform	ServiceNow
User Interface	ServiceNow Web UI
Authentication	ServiceNow Authentication
Authorization	ServiceNow ACL
Role Management	ServiceNow Users & Roles
Server-Side Logic	JavaScript
Database	ServiceNow Platform Database
Application Table	u_institution_details
Access Operations	READ, CREATE, WRITE, DELETE
Infrastructure	ServiceNow Cloud Platform

The project does not require a separate external database, machine-learning model, external API, file-storage service or separate cloud infrastructure.

---

4. PROJECT DESIGN

4.1 Problem Solution Fit

Aspect	Description
Target Users	ServiceNow administrators and authorized users
Problem	Role-based access alone may not provide sufficient record-level restrictions.
Proposed Solution	Combine ACLs with server-side scripts that validate Branch values.
Key Innovation	Field-value-based access control using scripted ACL evaluation.
Security Impact	Prevents unauthorized users from accessing restricted records.
Business Value	Provides configurable access control for enterprise ServiceNow applications.
Scalability	Additional branches, users, roles and rules can be incorporated.

4.2 Proposed Solution

The proposed solution uses ServiceNow ACLs to control access to the u_institution_details table.

Separate ACL configurations are used for:

- **READ**
- **CREATE**
- **WRITE**
- **DELETE**

The ACL first evaluates the user's role and configured conditions. For operations requiring branch-specific restrictions, a server-side JavaScript script validates the Branch value of the requested record.

If the user satisfies the required authorization and branch condition, the operation is allowed. Otherwise, ServiceNow denies access.

This provides a layered authorization mechanism:

User → Role → ACL → Condition → Branch Script Validation → Access Decision

4.3 Solution Architecture

Architecture Flow

User

↓

ServiceNow Web Interface

↓

Authentication

↓

Role Verification

↓

ServiceNow ACL Engine

↓

ACL Conditions

↓

Server-Side ACL Script

↓

Branch Validation

↓

Access Decision

↙ ↘

Allow Deny

↓ ↓

u_institution_details Access Denied

Major Components

1. **User** – Requests access to institution records.
2. **ServiceNow Web Interface** – Provides the application interface.
3. **Authentication** – Identifies the requesting user.
4. **User & Role Management** – Determines assigned roles.
5. **ACL Engine** – Evaluates READ/CREATE/WRITE/DELETE permissions.
6. **ACL Conditions** – Performs configured condition checks.
7. **ACL Script** – Executes server-side JavaScript.
8. **Branch Validation** – Checks the Branch value.
9. **Access Decision** – Grants or denies the requested operation.
10. **u_institution_details Table** – Stores the protected records.

---

5. PROJECT PLANNING & SCHEDULING

5.1 Project Planning

The project was divided into three development sprints.

Sprint 1 – Foundation

Story	Task	Story Points	Priority
USN-1	User login and identification	2	High
USN-2	Create and manage users	3	High
USN-3	Assign roles	3	High
USN-4	Create/configure institution table	3	High
USN-5	Configure Branch field	2	High

Sprint 1 Total: 13 Story Points

Sprint 2 – ACL Implementation

Story	Task	Story Points	Priority
USN-6	Configure READ ACL	3	High
USN-7	Configure CREATE ACL	2	High
USN-8	Configure WRITE ACL	2	High
USN-9	Configure DELETE ACL	2	High

Sprint 2 Total: 11 Story Points

Sprint 3 – Script Validation & Testing

Story	Task	Story Points	Priority
USN-10	Implement Branch validation script	3	High
USN-11	Restrict unauthorized branch records	3	High
USN-12	Perform complete ACL testing	3	High

Sprint 3 Total: 9 Story Points

Overall Project Plan

Sprint	Duration	Story Points
Sprint 1	05 Oct 2026 – 10 Oct 2026	13

Sprint 2	12 Oct 2026 – 17 Oct 2026	11
Sprint 3	19 Oct 2026 – 24 Oct 2026	9
Total	**18 working days**	**33**

Average Sprint Velocity: 11 Story Points/Sprint

Burndown Milestones: 33 → 20 → 9 → 0

---

6. FUNCTIONAL AND PERFORMANCE TESTING

6.1 Performance Testing

Performance testing was performed to verify that ACL evaluation and server-side script execution do not introduce significant delays during normal record operations.

The following operations were considered:

- User authentication
- Role verification
- READ ACL evaluation
- CREATE ACL evaluation
- WRITE ACL evaluation
- DELETE ACL evaluation
- Branch-based script validation
- Unauthorized access rejection

The ACL scripts were designed to perform only the required validation logic. Unnecessary database operations and complex processing were avoided to maintain efficient access decisions.

Functional Test Coverage

Test Area	**Test Cases**	**Expected Result**
User & Role Management	3	Correct users and roles identified
READ ACL	3	Authorized users can read
CREATE ACL	2	Authorized users can create
WRITE ACL	2	Authorized users can modify

DELETE ACL	2	Authorized users can delete
Branch Script Validation	3	Branch restriction correctly applied
Unauthorized Access	3	Unauthorized access denied
Final Validation	2	Complete ACL workflow verified

Total Test Cases: 20

---

7. RESULTS

7.1 Output Screenshots

The following screenshots should be included in the final report as evidence of implementation:

1. ServiceNow login screen
2. User administration screen
3. Role assignment screen
4. u_institution_details table
5. Institution record with Branch field
6. READ ACL configuration
7. CREATE ACL configuration
8. WRITE ACL configuration
9. DELETE ACL configuration
10. ACL server-side JavaScript script
11. Authorized user accessing a permitted record
12. Unauthorized user attempting to access a restricted branch record
13. Access-denied result
14. Successful record creation
15. Successful record modification
16. Successful/blocked record deletion

Result Summary

The implementation demonstrates that:

- Authorized users can perform permitted operations.
- ACLs correctly evaluate READ, CREATE, WRITE and DELETE permissions.
- Server-side scripts provide additional field-based authorization.
- Users attempting to access restricted Branch records are denied.
- The authorization logic is centrally managed within ServiceNow.

---

8. ADVANTAGES & DISADVANTAGES

Advantages

1. **Improved Security** – Prevents unauthorized access to restricted records.
2. **Field-Level Decision Making** – Access can depend on values such as Branch.
3. **Centralized Authorization** – Security rules are maintained through ServiceNow ACLs.
4. **Role-Based Control** – Existing ServiceNow roles can be integrated with the solution.
5. **Flexible Rules** – Server-side scripts can implement customized authorization logic.
6. **Scalable** – Additional branches and authorization rules can be introduced.
7. **Reduced Manual Control** – Access decisions are automatically evaluated by the platform.

Disadvantages

1. **Script Dependency** – Incorrect ACL scripts can result in unexpected access behavior.
2. **Configuration Complexity** – Multiple ACLs and conditions can become difficult to maintain.
3. **ServiceNow Dependency** – The solution depends on the ServiceNow platform.
4. **Testing Requirement** – Changes to ACL rules require thorough authorization testing.
5. **Administrator Expertise** – Effective implementation requires knowledge of ServiceNow ACLs and server-side scripting.

---

9. CONCLUSION

The project **“Script-Controlled ACL – Restrict Record Access Based on Field Value”** successfully demonstrates how ServiceNow Access Control Lists can be combined with server-side scripting to implement secure record-level authorization.

The system uses user roles, ACL conditions and JavaScript-based Branch validation to control READ, CREATE, WRITE and DELETE operations on the u_institution_details table.

Unlike simple role-based access control, the proposed approach can make authorization decisions according to the actual value of a record field. This makes the solution suitable for scenarios where users should only access records belonging to specific branches or organizational units.

The project demonstrates the importance of layered authorization and provides a practical approach for implementing customizable access control within the ServiceNow platform.

---

10. FUTURE SCOPE

The project can be further enhanced in the following areas:

1. **Multiple Branch Support** – Implement authorization for multiple branches and organizational units.
2. **Dynamic User-to-Branch Mapping** – Automatically determine the user's permitted branch from user or role information.
3. **Advanced Record-Level Security** – Introduce more complex conditions involving multiple fields.
4. **Audit and Monitoring** – Track denied access attempts and administrative ACL changes.
5. **Automated Testing** – Develop automated test cases for ACL validation.
6. **Security Dashboard** – Provide administrators with an overview of access decisions and security events.
7. **Hierarchical Authorization** – Support organization, department, branch and team-level access.
8. **Reusable ACL Framework** – Extend the same security model to other custom ServiceNow tables.
9. **Performance Optimization** – Optimize complex scripts for environments containing large numbers of records.
10. **Integration with Enterprise Identity Management** – Connect authorization rules with centralized enterprise identity and organizational information.

Team Members

Name	Responsibility
Pranaya S	Team Lead, ACL Configuration & Outcome
Deepika P	CREATE ACL
Sajitha K	Users & WRITE ACL
Thenmozhi R	Roles & DELETE ACL
Sharmila M	Table Creation & Configuration

End of Project Report